

Modos de operación y la línea de Coopaburrá

Cincuenta y cinco minutos en clase; los minutos de cada ejercicio son un ritmo sugerido y el docente gestiona el tiempo, así que lo que no se cierre en el aula continúa como trabajo independiente. Las demostraciones corren en su navegador, sin conexión y sin instalar nada. Los ejercicios 1 y 3 se hacen con OpenSSL o con Python en su máquina: los comandos están aquí, con botón de copiar.

Antes de empezar. No se evalúa que los comandos funcionen. Se evalúa que expliquen **qué propiedad del modo permitió —o impidió— el ataque**. Un ejercicio que corrió sin explicación vale la mitad que uno que falló con el diagnóstico correcto.

¿En qué terminal? En Windows, los comandos de abajo van en **Git Bash** (viene con Git para Windows y trae OpenSSL y las órdenes head, tail y cat), **no en cmd ni en PowerShell**, donde no existen. En macOS o Linux, la Terminal normal. La alternativa en Python corre en cualquier terminal.

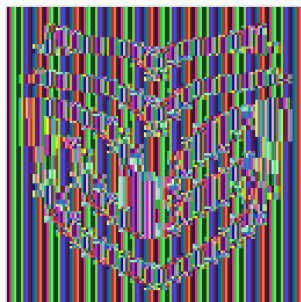
DEMOSTRACIÓN • 5 MINUTOS

Lo que ECB no esconde

El escudo de la Institución, aplanado a dos colores y cifrado con **AES-256** y una llave de 256 bits generada al azar. Se cifraron los bytes de los píxeles, no el archivo: por eso el resultado sigue siendo una imagen. Lo único que cambia entre las dos versiones cifradas es el modo de operación.



ORIGINAL
SIN CIFRAR



AES-256
MODO ECB



AES-256
MODO CTR

Y ahora con datos, que es donde duele

Una imagen impresiona, pero el problema real está en los registros. Este texto tiene cuatro bloques de dieciséis caracteres y **tres de ellos son idénticos**. Cífrelo en los dos modos y compare bloque contra bloque.

UN REGISTRO CON VALORES REPETIDOS

```
CUOTA=000100000 CUOTA=000100000 CUOTA=000100000 CUOTA=000900000
```

La llave es la misma en los dos casos

MODO ECB • BLOQUE A BLOQUE

MODO CBC • BLOQUE A BLOQUE

Los bloques que aparecen **en naranja** están repetidos dentro del mismo texto cifrado.

Panel interactivo • ábralo en el navegador para ejecutarlo

Por qué pasa esto, y por qué importa más en una base de datos que en una foto

ECB cifra cada bloque de dieciséis bytes por separado y siempre de la misma forma. Bloques de entrada iguales producen bloques de salida iguales, así que **la estructura del mensaje sobrevive al cifrado**. En la imagen eso dibuja el escudo; en una tabla permite contar cuántos registros comparten un valor, y a partir de ahí deducir cuál es sin descifrar nada.

CBC encadena cada bloque con el anterior y arranca de un vector de inicialización aleatorio, así que dos bloques iguales producen salidas distintas. CTR hace lo propio combinando cada bloque con un contador distinto. Ninguno de los dos, sin embargo, **detecta** una alteración: eso es lo que van a comprobar en el ejercicio 2.

EJERCICIO 1 • 15 MINUTOS

Reproducir el escudo

Tomen una imagen con zonas de color uniforme, cifrenla en ECB y en CBC conservando la cabecera del archivo, y comparen. Entreguen las dos imágenes y la explicación de la diferencia.

- 1 Usen el BMP de muestra `escudo-plano.bmp`, que viene con el laboratorio**
 - 24 bits, colores planos y 128 px de ancho, para que los píxeles queden en un múltiplo de 16 bytes y -nopad no dé error. Si prefieren traer la suya, que sea un BMP de 24 bits con pocos colores y zonas grandes de un solo tono: con degradados o fotografía el efecto no aparece, porque ECB solo delata lo que se repite.
- 2 Separen la cabecera de los píxeles.**

En un BMP la cabecera ocupa los primeros 54 bytes. Si la cifran también, el archivo deja de abrirse.
- 3 Cifren solo los píxeles**

, en ECB y en CBC, con la misma llave y sin relleno.

4

Vuelvan a pegar la cabecera

delante de cada resultado y abran los dos archivos.

CON OPENSSL • LA LLAVE EN HEXADECIMAL, NO UNA CONTRASEÑA

```
# llave de 256 bits y vector de 128, como el martes
openssl rand -hex 32 > llave.hex
openssl rand -hex 16 > vi.hex

# separar cabecera (54 bytes) y píxeles
head -c 54 escudo-plano.bmp > cabecera.bin
tail -c +55 escudo-plano.bmp > pixeles.bin

# cifrar SOLO los píxeles, sin relleno, con -K mayúscula
openssl enc -aes-256-ecb -in pixeles.bin -out ecb.bin -K $(cat llave.hex) -nopad
openssl enc -aes-256-cbc -in pixeles.bin -out cbc.bin -K $(cat llave.hex) -iv $(cat vi.hex) -nopad

# volver a armar las imágenes
cat cabecera.bin ecb.bin > escudo-ecb.bmp
cat cabecera.bin cbc.bin > escudo-cbc.bmp
```

O CON PYTHON, QUE ADEMÁS LES DEJA MEDIR

```
# pip install pillow cryptography
import os
from PIL import Image
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

img = Image.open("escudo-plano.bmp").convert("RGB")
img = img.quantize(colors=2).convert("RGB") # aplanar: sin esto no se ve nada
crudo = img.tobytes()
assert (img.width * 3) % 16 == 0 # la fila debe cuadrar con el bloque

llave, vi = os.urandom(32), os.urandom(16)
def cifra(modo):
    c = Cipher(algorithms.AES(llave), modo).encryptor()
    return c.update(crudo) + c.finalize()

Image.frombytes("RGB", img.size, cifra(modes.ECB())[ :len(crudo)]).save("ecb.png")
Image.frombytes("RGB", img.size, cifra(modes.CBC(vi))[ :len(crudo)]).save("cbc.png")
```

Si no se ve el patrón, revisen tres cosas en este orden: que la imagen esté aplanada a pocos colores, que el ancho en bytes sea múltiplo de 16 —de lo contrario cada fila arranca desfasada y el dibujo se desalinea— y que hayan usado **-K mayúscula** con la llave en hexadecimal y no **-k** minúscula con una contraseña. Esa diferencia es el ejercicio 3.

QUÉ SE ENTREGA DE ESTE EJERCICIO

- Las dos imágenes cifradas.
- Tres o cuatro frases explicando por qué una deja ver el dibujo y la otra no. No basta con decir «porque ECB es inseguro»: hay que decir qué propiedad lo hace visible.
- Una frase sobre dónde vieron ustedes ese mismo problema en Coopaburrá, con el sistema y el hallazgo.

EJERCICIO 2 • 20 MINUTOS

Alterar un byte y ver quién se da cuenta

Cifren un texto en **CTR** y en **GCM**, alteren un byte del texto cifrado en los dos casos e intenten descifrar. Documenten qué pasa en cada modo y por qué. Este panel hace el ataque completo en su navegador; el mismo ataque en Python está más abajo.

EL ATAQUE

MENSAJE ORIGINAL

VALOR=0000100000

BYTE NÚMERO

6

ALTERAR CON

09

El byte 6 del mensaje de ejemplo es el primer cero del valor. Combinarlo con 09 lo convierte en un nueve: novecientos mil pesos en lugar de cien mil, sin conocer la llave.

Panel interactivo • ábralo en el navegador para ejecutarlo

Qué acaba de pasar, en detalle

En **CTR** el cifrado consiste en combinar el mensaje con una secuencia que depende de la llave y de un contador. Esa combinación es reversible byte a byte: cambiar un byte del texto cifrado cambia **exactamente ese byte** del texto en claro, y ningún otro. El atacante no necesita la llave para saber qué va a salir; le basta saber qué decía el original en esa posición. Por eso la salida no es basura: es el mensaje que el atacante eligió.

En **GCM** el cifrado es el mismo —GCM usa CTR por dentro—, pero además calcula una etiqueta de autenticación sobre el texto cifrado y sobre los datos asociados. Al descifrar, la etiqueta se recalcula y se compara. Si no coincide, la librería **no devuelve el texto**: lanza un error. No devuelve el texto alterado, ni el texto a medias: no devuelve nada. Esa es la diferencia entre cifrar y proteger.

Los datos asociados del ejemplo —el número de cuenta— no van cifrados, pero sí entran en la etiqueta. Sirven para amarrar el mensaje a su contexto: un texto cifrado válido para la cuenta 4471 no se puede reutilizar en otra cuenta, porque la etiqueta ya no cuadra.

EL MISMO EJERCICIO EN PYTHON

```
# pip install cryptography
import os
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

llave = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12) # 96 bits, distinto cada vez
c = AESGCM(llave).encrypt(nonce, b"VALOR=0000100000", b"cuanta=4471")

alterado = bytearray(c); alterado[6] ^= 0x09 # el ataque de arriba
AESGCM(llave).decrypt(nonce, bytes(alterado), b"cuanta=4471") # → InvalidTag
```

QUÉ SE ENTREGA DE ESTE EJERCICIO

- Las dos salidas: la de CTR, con el valor cambiado, y la de GCM, con el error.
- La explicación de por qué CTR permite elegir el resultado del ataque sin conocer la llave.
- **La pregunta que hay que responder sí o sí:** ¿qué pasaría si el mismo nonce se usara en dos registros distintos con la misma llave? Respondan para CTR y para GCM por separado, porque no es lo mismo.

EJERCICIO 3 · 15 MINUTOS

Auditar la evidencia C

Esta es la línea real del programador de tareas de Coopaburrá: la que produce el respaldo diario que viaja a Sabaneta los viernes. Tiene **tres problemas distintos** y solo uno es evidente.

```
openssl enc -aes-256-cbc -in respaldo_diario.sql -out respaldo_diario.enc \
-k $(cat /opt/backup/llave.txt)
```

Herramienta · qué hace realmente cada opción

Las herramientas ofrecen dos formas de darles una llave y no son lo mismo, aunque se parezcan en la línea de comandos. Escriba una contraseña y compare.

DERIVACIÓN DE LLAVE

llave-de-respaldos-2019

La misma contraseña produce siempre la misma llave. Lo que cambia es cuánto le cuesta a un atacante probar la siguiente.

Panel interactivo · ábralo en el navegador para ejecutarlo

-K mayúscula

Recibe la llave en hexadecimal: exactamente los bits que se van a usar. Es lo correcto cuando ya se tiene una llave generada al azar.

-k minúscula

Recibe una contraseña de texto y la pasa por una función de derivación. La calidad depende de esa función y de sus parámetros.

Lo que hace el script del caso

Usa la minúscula con el contenido de un archivo. Trata una llave que ya era aleatoria como si fuera una contraseña.

La herramienta lo está avisando. Al correr esa línea, OpenSSL imprime en pantalla: ***** WARNING : deprecated key derivation used. Using -iter or -pbkdf2 would be better.** Está ahí desde 2018, cada noche, entre las 11:40 y las 2:15. Nadie lee la salida de un proceso programado.

Su hoja de auditoría

Escriban los tres problemas. Este panel no los corrige: es para ordenar el trabajo antes de pasarlo al informe.

PROBLEMA A • EL EVIDENTE

Qué está mal y qué consecuencia tiene

PROBLEMA B • SOBRE CÓMO SE DERIVA LA LLAVE

Qué está mal y qué consecuencia tiene

PROBLEMA C • SOBRE QUÉ HACE, Y QUÉ NO HACE, EL MODO UTILIZADO

Qué está mal y qué consecuencia tiene

LA LÍNEA CORREGIDA

Escríbanla completa, con todas sus opciones

Se descarga a su equipo. Nada sale del navegador.

Panel interactivo • ábralo en el navegador para ejecutarlo

Cómo saber si van bien encaminados

No son pistas de la respuesta, son las preguntas que la respuesta tiene que contestar. **Sobre la llave:** ¿quién puede leerla hoy, y en cuántos sitios distintos está? Cuenten los sitios; son más de uno. **Sobre la derivación:** si la llave ya venía de un generador aleatorio, ¿qué le aporta pasarla por una función pensada para contraseñas humanas, y qué le quita? **Sobre el modo:** si alguien modifica el archivo de respaldo mientras viaja en el carro del mensajero, ¿en qué momento lo notaría Coopaburrá, y cómo?

Y para la línea corregida, tres preguntas más: ¿de dónde sale la llave?, ¿cómo se la entregan a la herramienta sin que aparezca en la lista de procesos?, y ¿qué tendría que pasar al restaurar para que se sepa que el archivo llegó íntegro?

REPASO • 5 MINUTOS

¿Qué modo es el del caso?

Seis situaciones. En una de ellas la respuesta correcta es que ningún modo sirve, porque el problema no es el modo.

1. El resultado del cifrado deja ver la silueta de lo que se cifró.

2. Se altera un byte del texto cifrado y el descifrado devuelve el mensaje con exactamente ese byte cambiado, sin ningún error.

3. Se altera un byte del texto cifrado y el bloque correspondiente sale convertido en basura, mientras el siguiente cambia solo en ese byte.

4. Hay que cifrar el respaldo y, además, poder demostrar al restaurarlo que nadie lo modificó en el camino.

5. Repetir el número de un solo uso con la misma llave no solo expone los dos mensajes: además permite falsificar la etiqueta de autenticación.

6. Hace falta cifrar un campo de la base de datos de modo que dos registros con el mismo valor produzcan el mismo resultado, para poder agruparlos.

CIERRE

Qué se entrega y cómo se califica

Una página por equipo al final del bloque. Se evalúa el análisis, no el resultado.

- ☐ Las dos imágenes del ejercicio 1, con la explicación de qué propiedad de ECB hace visible el dibujo.
- ☐ Las dos salidas del ejercicio 2 y la respuesta sobre el nonce repetido, separada para CTR y para GCM.
- ☐ Los tres problemas de la evidencia C, cada uno con su consecuencia para Coopaburrá.
- ☐ La línea corregida, completa.
- ☐ Una frase final: de los hallazgos H3 y H4 del caso, ¿cuál de los dos quedaría resuelto con lo que aprendieron hoy, y cuál no?

Regla de trabajo del módulo. No carguen el caso, las evidencias ni sus respuestas en servicios en línea de terceros. Es la regla 5 del caso y, además, lo que se pega en un sitio público deja de ser confidencial en ese instante.